

Implementation of RSA

Supervisor,
Dr. Narendran Rajagopalan,
Associate Professor,
NITPY.

Assignment by,
Sampangi Nagavardhan
(CS22B1049)

Overview:

The RSA cryptosystem, developed by Rivest, Shamir, and Adleman, is among the most widely adopted public-key encryption methods. It underpins secure communication over the internet by enabling encryption and digital signatures through asymmetric key pairs. By leveraging mathematically related public and private keys, RSA provides data confidentiality, authenticity, and integrity.

Methodology:

○ Key Generation

- Choose two large prime numbers p and q .
- Compute $n = p \times q$ (modulus).
- Calculate Euler's totient: $\phi(n) = (p-1)(q-1)$.
- Choose an encryption exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
- Compute decryption exponent d as the modular inverse of $e \bmod \phi(n)$.

○ Encryption

- Convert the plaintext message to an integer m , such that $m < n$
- Compute ciphertext: $c = m^e \bmod n$

○ Decryption

- Recover plaintext: $m = c^d \bmod n$

Applications:

- RSA is often used for secure key exchange and authentication in VPNs.
- Developers use RSA to sign software and updates, ensuring that code has not been tampered with and is from a trusted source.
- Secure Email Communication (e.g., PGP)
- Digital Signatures for authentication
- TLS/SSL protocols for HTTPS websites
- Software licensing and code signing

Keyspace & Security Complexity:

- **Keyspace Size:** Determined by the size of the modulus n ; modern RSA implementations typically use key lengths ranging from 2048 to 4096 bits.
- **Security Basis:** Depends on the computational difficulty of factoring large semi-prime numbers.
- **Complexity:** Factoring a 2048-bit RSA modulus using classical methods (e.g., the General Number Field Sieve) has sub-exponential time complexity, making it infeasible with current technology.
- **Quantum Vulnerability:** While RSA remains secure against classical attacks, it is theoretically breakable by **Shor's algorithm** on a sufficiently powerful quantum computer.

Result :

```
C:\Users\Nagavardhan\Downloads>C:/Python313/python.exe c:/Users/Nagavardhan/Downloads/RSA_CS22B1049_Implementation.py
Welcome to the RSA Encryption Program!

RSA Key Pair Generated!
Public Key (e, n): (159830178605, 682312867253)
Private Key (d, n): (193664120381, 682312867253)

Please enter the message you want to encrypt (plaintext):
RSA is theoretically vulnerable to quantum attacks (e.g., Shor's algorithm).Use padding schemes like OAEP in production for added security.

Encrypted message (ciphertext):
[78271830009, 357720363567, 457330365443, 142235693077, 81779906733, 658078720484, 212590194197, 240738230044, 591533397650, 611650111286, 390170297
701, 145472887562, 514789250423, 129215697381, 5111884076, 285211796325, 144500384024, 291586862391, 90567606269, 107636225358, 242484116035, 2707066
78626, 303601939453, 597394064357, 441156982939, 624338803859, 102605829732, 461462980786, 119272337545, 415953933600, 305454807366, 120150325121, 55
303331446, 658065907439, 151491166153]

Decrypted message:
RSA is theoretically vulnerable to quantum attacks (e.g., Shor's algorithm).Use padding schemes like OAEP in production for added security.
```

Conclusion :

RSA continues to be a foundational element of modern cryptography, supporting secure data transmission and authentication in digital systems. Its reliability stems from asymmetric key encryption and the mathematical challenge of factoring large prime products. Although post-quantum cryptographic methods are emerging, RSA remains a widely trusted and integral part of current digital security frameworks.